

Лабораторная работа №19. Введение в JavaScript. Сравнение с C#

Цель работы:

- Познакомиться с языком программирования JavaScript;
- Понять его место в web-разработке;
- Сравнить базовые концепции JavaScript и C#;
- Научиться запускать JavaScript-код в браузере и через Node.js.

Шаг 1. Подготовка рабочего окружения

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:
Lab19_LearnJs_FIO (где FIO — ваша фамилия)
5. Перейдите в созданную директорию
6. Создайте структуру проекта:
 - **README.md**
 - **index.html**
 - **main.js**

Откройте созданную папку в **Visual Studio Code**

Шаг 2. Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:
git init
2. Добавьте файлы в индекс:
git add .
3. Создайте первый коммит:
git commit -m "Initial commit"

Шаг 3. Создание удалённого репозитория

1. Создайте **пустой public-репозиторий** на GitHub с именем:
Lab19_LearnJs
2. Не добавляйте:
 - **README**
 - **.gitignore**
 - **License**

3. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

4. Отправьте файлы:

git push -u origin main

5. Сделайте **скриншот терминала** с выполненными командами:

- commit
- remote add
- push

6. Создайте папку **img** и сохраните в ней изображение удачного пуша под именем:

gitPushLab19_FIO.png

7. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

git push

Шаг 4. Подключение JavaScript к HTML

1. Откройте файл **index.html** и добавьте базовую **HTML**-структуру.

2. Подключите **script** в тег **body**.

Шаг 5. Циклы в JavaScript

5.1. Теоретическая часть

Циклы в JavaScript по синтаксису почти полностью совпадают с C#.

В JavaScript используются циклы:

- **for**
- **while**
- **do...while**

5.2. Цикл for

Базовый пример (обязателен к переписыванию):

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Отличие только в отсутствии типа int.

5.3. Практическое задание №1

Задание:

1. Используя цикл **for**, выведите числа от **1** до **10**
2. Выведите только **чётные** числа
3. Подсчитайте сумму чисел от **1** до **10**

5.4. Цикл while

Пример (обязателен к переписыванию):

```
let count = 0;

while (count < 3) {
  console.log("Count:", count);
  count++;
}
```

5.5. Практическое задание №2

Задание:

1. Создайте переменную number = 5
2. Используя while, уменьшайте значение на 1
3. Выведите все значения до 0

5.6. Цикл do...while

Цикл do...while гарантирует хотя бы одно выполнение тела цикла.

Пример (обязателен к переписыванию):

```
let doValue = 0;
do {
  console.log("Value:", doValue);
  doValue++;
} while (doValue < 3);
```

5.7. Операторы break и continue

Пример break:

```
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    break;
  }
  console.log(i);
}
```

Пример continue:

Поведение идентично C#.

5.8. Практическое задание №4

Задание:

1. Используя for, выведите числа от 1 до 10
2. Пропустите число 5
3. Прекратите цикл при достижении 8

5.9. Вложенные циклы

Пример (обязателен к переписыванию):

```
for (let i = 1; i <= 3; i++) {  
  for (let j = 1; j <= 3; j++) {  
    console.log(`i = ${i}, j = ${j}`);  
  }  
}
```

Используется для:

- таблиц
- матриц
- перебора комбинаций

5.10. Практическое задание №5

Задание:

Используя вложенные циклы, выведите:

*

**

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:
step5_loopsLab19_FIO.png
2. Выполните следующие команды:

git add .

git commit -m "Step 5: loops in JavaScript"

git push

Шаг 6. Функции в JavaScript

6.1. Теоретическая часть

Функция в JavaScript — это блок кода, который можно вызывать многократно.

В C# используются методы, в JavaScript — функции, но по смыслу это одно и то же.

Основные отличия:

- в JS нет указания типа возвращаемого значения;
- параметры не имеют типов;
- функция может быть объявлена несколькими способами.

6.2. Объявление функции (классический способ)

Базовый пример (обязателен к переписыванию):

```
function sum(a, b) {  
  return a + b;  
}  
  
console.log(sum(3, 5));
```

JavaScript сам определяет тип возвращаемого значения.

6.3. Практическое задание №1

Задание:

1. Создайте функцию **multiply**
2. Функция принимает два параметра
3. Возвращает их произведение
4. Вызовите функцию и выведите результат в консоль

6.4. Функция без return

Функции могут ничего не возвращать.

Пример:

```
function sayHello(name) {  
  console.log(`Hello, ${name}`);  
}  
  
sayHello("Тимофей");
```

6.5. Практическое задание №2

Задание:

1. Создайте функцию **printInfo()**, которая принимает:
 - **имя**
 - **возраст**
2. **Выводит** информацию в **консоль**

6.6. Значения параметров по умолчанию

В JavaScript можно задавать значения по умолчанию.

Пример:

```
function greet(name = "Гость") {  
  console.log("Привет, " + name);  
}  
greet();  
greet("Анастасия");
```

В C# это появилось позже и используется реже.

6.7. Практическое задание №3

Задание:

1. Создайте функцию **calculateDiscount()**
2. Принимает **цену** и **процент скидки**
3. Если процент не передан — считать его **равным 10**
4. **Вернуть** итоговую цену

6.8. Функции как значения

В JavaScript функции — это значения, их можно:

- присваивать переменным
- передавать как аргументы

Пример:

```
const add = function(a, b) {  
  return a + b;  
};  
console.log(add(2, 3));
```

 Функции как значения — мощный инструмент (это как делегаты в C#)

Пример замыканий (closure):

```
function makeCounter() {  
  let count = 0; // Локальная переменная  
  return function () {  
    count++; // Функция "помнит" count  
    return count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2  
console.log(counter()); // 3
```

count "живёт" внутри функции!

6.9. Стрелочные функции (Arrow Functions)

Это современный и короткий способ создания функций.

Синтаксис:

Обычная функция:

```
function sumFunc(a, b) {  
  return a + b;  
}
```

Стрелочная функция:

```
const sumFunc2 = (a, b) => a + b;
```

Правила:

1. Один параметр — скобки необязательны:

```
const double = x => x * 2;
```

2. Несколько параметров — скобки обязательны:

```
const sumFunc3 = (a, b) => a + b;
```

3. Нет параметров — пустые скобки:

```
const sayGreeting = () => console.log("Hello");
```

4. Одна строка — `return` неявный:

```
const square = x => x * x; // return автоматически
```

5. Несколько строк — фигурные скобки + явный return:

```
const calculate = (a, b) => {  
  let result = a + b;  
  return result * 2;  
};
```

Сравнение синтаксисов:

Обычная функция	Стрелочная функция
function add(a, b) { ... }	const add = (a, b) => { ... }
function(x) { return x * 2; }	x => x * 2

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:

step6_functionsLab19_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 6: functions in JavaScript"

git push

Шаг 7. Массивы в JavaScript

7.1. Теоретическая часть

Массив в JavaScript — это упорядоченная коллекция элементов.

Отличия от C#:

- массив может содержать элементы разных типов
- размер массива не фиксирован
- массив — это объект

7.2. Создание массива

Базовый пример (обязателен к переписыванию):

```
let numbersArr = [1, 2, 3, 4, 5];  
console.log(numbersArr);
```

В JS нет указания типа элементов.

7.3. Доступ к элементам массива

```
console.log(numbersArr[0]);  
console.log(numbersArr[1]);
```

7.4. Практическое задание №1

Задание:

1. Создайте массив **colors** из **3** строк
2. Выведите **первый** и **последний** элемент
3. Измените **второй** элемент массива
4. Выведите массив **полностью**

7.5. Длина массива

```
console.log(numbersArr.length);
```

7.6. Добавление и удаление элементов

Добавление в конец (push)

```
numbersArr.push(10);  
console.log(numbersArr);
```

Удаление из конца (pop)

```
numbersArr.pop();  
console.log(numbersArr);
```

7.7. Практическое задание №2

Задание:

1. Создайте **пустой** массив **students**
2. Добавьте в массив **3** имени
3. Удалите **последний** элемент
4. Выведите **итоговый** массив

7.8. Перебор массива с помощью цикла for

```
let numbers2 = [10, 20, 30];  
for (let i = 0; i < numbers2.length; i++) {  
  console.log(numbers2[i]);  
}
```

7.9. Цикл for...of

Аналог *foreach* в C#

```
for (let value of numbers2) {  
  console.log(value);  
}
```

for...of — предпочтительный способ перебора массива.

7.10. Массивы с разными типами данных

```
let mixedArray = [1, "text", true, 3.14];  
console.log(mixedArray);
```

В C# такое возможно только через *object*.

7.11. Поиск элемента в массиве

Метод 1: `indexOf()` — возвращает индекс

```
console.log(numbersArr.includes(1));  
console.log(numbersArr.indexOf(2));
```

Если элемент не найден → возвращает -1

Метод 2: `includes()` — возвращает `true/false`

```
console.log(fruits.includes("яблоко")); // true  
console.log(fruits.includes("манго")); // false
```

Проще для проверки "есть ли элемент"

7.12. Практическое задание №4

Задание:

1. Создайте массив городов
2. Проверьте, есть ли в массиве заданный город
3. Если **есть** — выведите его индекс

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:

step7_arraysLab19_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 7: arrays in JavaScript"

git push

Шаг 8. Объекты в JavaScript

8.1. Теоретическая часть

Объект в JavaScript — это набор свойств в формате **ключ** → **значение**.

В JavaScript:

- объект можно создать без класса
- свойства можно добавлять на лету
- объект — это основная структура данных

Во фронтенде 90% данных — это объекты.

8.2. Создание объекта (object literal)

Базовый пример (обязателен к переписыванию):

```
let user2 = {  
  name: "Ivan",  
  age: 20,  
  isStudent: true,  
};  
  
console.log(user2);
```

В JS не нужен класс для создания объекта.

8.3. Доступ к свойствам объекта

Через точку:

```
console.log(user2.name);  
console.log(user2.age);
```

Через квадратные скобки:

```
console.log(user2["name"]);
```

Второй способ полезен, когда имя свойства хранится в переменной.

8.4. Практическое задание №1

Задание:

1. Создайте объект **book**
2. Свойства:
 - **title**
 - **author**
 - **year**

3. Выведите каждое свойство в консоль
4. Измените год издания

8.5. Добавление и удаление свойств

```
user2.age = 30;  
user2.name = "Кирилл";  
delete user2.isStudent;  
console.log(user2);
```

В C# такое невозможно без изменения класса.

8.6. Объект с методами

Добавим в уже имеющийся объект метод **sayHello()**:

```
let user2 = {  
  name: "Ivan",  
  age: 20,  
  isStudent: true,  
  sayHello: function () {  
    console.log(`Hello, my name is ${name}`);  
  },  
};  
  
user2.sayHello();
```

8.7. Практическое задание №2

Задание:

1. Создайте объект **car**
2. Свойства:
 - **brand**
 - **year**
3. Метод **getInfo()**, который выводит информацию об автомобиле

8.8. Перебор свойств объекта

```
for (let key in user2) {  
  console.log(key + ": " + user2[key]);  
}
```

Используется для:

- вывода данных;

- сериализации;
- отладки.

8.9. Практическое задание №3

Задание:

1. Создайте объект **product**
2. Добавьте несколько **свойств**
3. Выведите все свойства с помощью **for...in**

8.10. Вложенные объекты и массивы

```
let student = {  
  name: "Григорий",  
  skills: ["HTML", "CSS", "JS"],  
  address: {  
    city: "Волжский",  
    street: "Пушкина",  
  },  
};  
  
console.log(student.skills[0]);  
console.log(student.address.city);
```

Это типичная структура данных, приходящая с backend.

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:
step8_objectsLab19_FIO.png
2. Выполните следующие команды:

git add .

git commit -m "Step 8: objects in JavaScript"

git push

Финальный коммит (после всех шагов)

git add .

git commit -m "Completed Lab 18: JavaScript basics"

git push

Самостоятельное задание

Цель задания: Создать подробное и структурированное описание лабораторной работы №19 в файле README.md, используя Markdown.

Что должно быть в README.md

1. Заголовок

2. Основная информация

****ФИО:** Иванов Иван Иванович**

****Группа:** ИСП-231**

****Дата:** 18.03.2026**

3. Краткое описание работы

Описание (что изучили)

4. Структура проекта

Структура проекта

После сделайте в терминале:

git add .

git commit -m "Added complete README for Lab19"

git push